NoLibMath framework

Unsigned Variables & Math theorems

By BENJAMIN LOCKETT

1 Rationale

The aim of this project is to demonstrate how mathematical calculations can be broken into small, understandable operations in C++. By separating input, checking, calculation and output, each stage can be followed without relying on an external mathematics library. This also makes it easier to identify where a result becomes invalid.

The framework starts with unsigned integer powers, then applies the same structure to the expression parser and the supporting mathematics. The original library is retained. New parsing and interface code sit around it, allowing a user to type a calculation directly instead of entering coefficients one at a time.

1.1 What unsigned means

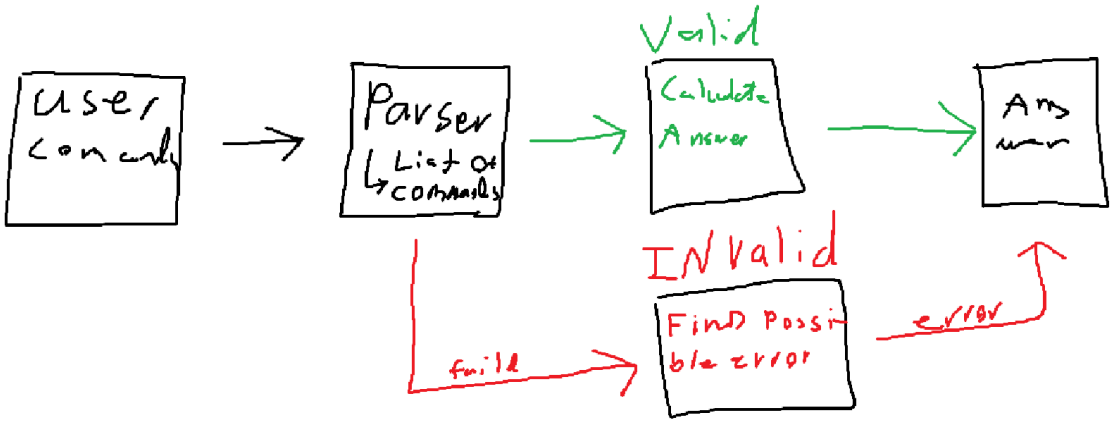
An unsigned integer stores whole numbers from zero upward. It cannot store a negative sign or a decimal part. In this Visual Studio build, ULong is a 64-bit unsigned integer, so its largest possible value is:

$$M = 2^{64} - 1 = 18\ 446\ 744\ 073\ 709\ 551\ 615 \quad (1)$$

Unsigned storage is used for values such as an integer base, an exponent magnitude and a node index. Other calculations use SLong for signed integers and Real for approximate real values. Therefore, the project is not restricted to positive answers simply because some of its storage is unsigned.

1.2 Design pipeline

The operation is selected first. Its input is then checked before the mathematics is allowed to run. If a stage fails, the program returns an explanation rather than treating an invalid value as a result.



2 Unsigned checks and integer powers

The main issue with unsigned arithmetic is that a calculation can exceed the storage limit. A wrapped value may still look like a valid number. To prevent this, the code checks whether the next operation will fit before performing it.

2.1 Reading the digits

Let v be the value already read and d the next digit. Applying decimal place value gives the next value. Rearranging against the maximum M gives the check used by readUnsigned:

$$v_{next} = 10v + d, \qquad v \le \left\lfloor \frac{M - d}{10} \right\rfloor \tag{2}$$

Only digits are accepted on this unsigned input path. A minus sign, a decimal point or a value above M is rejected. The expression parser is a separate path and can accept signed and decimal values.

2.2 Checking the operation

For unsigned a and b, addition and multiplication are permitted only when the following conditions hold. A zero multiplication is handled directly, avoiding division by zero in the check.

$$a + b \le M \quad \Leftrightarrow \quad a \le M - b \tag{3}$$

$$ab \le M \quad \Leftrightarrow \quad b \le \left\lfloor \frac{M}{a} \right\rfloor, \qquad a > 0 \tag{4}$$

2.3 Exact result, then scientific result

For a base b and exponent n, largestExactPower starts at 1 and multiplies by b while the next result fits. If it reaches n, the answer is exact. Otherwise, it reports the last exact power and uses scientificPower for an approximate final result.

$$p_0 = 1, \qquad p_{k+1} = b\,p_k \qquad while \qquad p_k \le \left\lfloor \frac{M}{b} \right\rfloor \tag{5}$$

Substituting b = 2 shows the boundary: the power with exponent 63 fits, while exponent 64 does not. This is a storage limit, not a failure of the mathematical operation. The cases with base zero or one, and exponent zero, are handled separately. This project rejects zero raised to zero.

3 Scientific representation and parsing

3.1 Keeping the scale separate

A scientific value stores a mantissa m and an exponent magnitude p separately. A Boolean flag records whether the decimal exponent is negative. For a nonzero value, normalisation keeps the mantissa within the following range:

$$V = m \times 10^{sp}, \qquad 1 \le |m| < 10, \qquad s \in \{-1, 1\} \tag{6}$$

Applying multiplication means multiplying the mantissas and adding the signed exponents. If the exponent signs differ, subtract the smaller magnitude from the larger and keep the larger magnitude’s sign. The exponent magnitude must still fit inside ULong.

$$\left(m_1 \times 10^{e_1}\right)\left(m_2 \times 10^{e_2}\right) = m_1 m_2 \times 10^{e_1 + e_2} \tag{7}$$

scientificPower uses repeated squaring: multiply the running result when the exponent is odd, halve the exponent, then square the factor when more work remains. The result retains a large scale, but the mantissa remains approximate; this is not unlimited exact precision.

$$b^{2k} = (b^k)^2, \qquad b^{2k+1} = b(b^k)^2 \tag{8}$$

3.2 Understanding keyboard input

```
action[dirive(x^3 + 2x + 1)]
```

The command selects differentiation. The parser reads the contents of the parentheses as numbers, a variable, operators and grouped expressions. It recognises 2x as multiplication and creates nodes that record how the expression is connected. The unsigned node indices identify those nodes; they are not the mathematical answer.

$$f(x) = x^3 + 2x + 1 \tag{9}$$

Applying operator precedence means powers are grouped before multiplication and addition. A leading minus applies after a power, while chained powers associate to the right. Parentheses can change either grouping.

$$-x^2 = -(x^2), \qquad 2^{3^2} = 2^9 = 512 \tag{10}$$

A successful parse must consume the whole input. Missing brackets, incomplete powers and unknown names return an error with a caret at the relevant column. The next line can then be entered without restarting the program.

4 Calculus from the parsed expression

Once the expression has been identified, the calculation follows the operation stored in each node. For a polynomial, the coefficients can be collected by power. Differentiation then acts on each term, rather than treating the input as an unstructured line of text.

$$P(x) = \sum_{k=0}^{n} a_k x^k, \qquad P'(x) = \sum_{k=1}^{n} k\, a_k x^{k-1} \tag{11}$$

Substituting the example expression into the derivative rule:

$$f'(x) = 3x^2 + 2 + 0 = 3x^2 + 2 \tag{12}$$

Evaluation uses the same expression with a supplied value of x. For x = 2:

$$f(2) = 2^3 + 2(2) + 1 = 13 \tag{13}$$

4.1 Products, quotients and functions

The symbolic routines also apply product, quotient and chain rules. Here u and v are expressions in x, and g is a supported function. These rules allow the derivative to follow the structure of a nested expression.

$$(uv)' = u'v + uv', \qquad \left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2} \tag{14}$$

$$[g(u)]' = g'(u)u', \qquad [\sin(x^2)]' = 2x\cos(x^2) \tag{15}$$

4.2 Reversing the calculation

Integration increases a polynomial power by one and divides by the new power. The integration constant C is added to the displayed result. For the derivative above:

$$\int (3x^2 + 2)\, dx = x^3 + 2x + C \tag{16}$$

The non-polynomial integrator uses a restricted set of rules. A candidate is differentiated and compared with the input at sample points using guarded evaluation. Passing these checks supports the result, but does not prove a symbolic identity. If the rule is unsupported or the check fails, no candidate is accepted.

The current workbench accepts one variable, integer powers, pi, e, and sin, cos, tan, atan, exp, ln and lnabs. Angles are in radians. ln requires a positive argument; lnabs means the logarithm of the absolute value and requires a nonzero argument.

5 Supporting mathematical routines

The retained library also contains the following calculations. These formulas explain their main logic; they are not all available as text commands in the new workbench. Real and complex results use floating-point arithmetic.

5.1 Roots, exponentials and logarithms

The square-root routine improves a positive guess repeatedly. A positive-base real power is calculated through a logarithm and an exponential. The exponential uses a reduced argument r and a 64-term update after the initial term:

$$g_{k+1} = \frac{1}{2}\left(g_k + \frac{a}{g_k}\right), \qquad a^q = \exp(q \ln a), \qquad a > 0 \tag{17}$$

$$x = k \ln 2 + r, \qquad \exp(x) \approx 2^k \sum_{j=0}^{64} \frac{r^j}{j!} \tag{18}$$

For the logarithm, the code scales a positive input into the interval from 0.75 to 1.5, then applies a series. The factor removed by scaling is restored using k ln 2:

$$a = y2^k, \qquad z = \frac{y - 1}{y + 1}, \qquad \ln a \approx k \ln 2 + 2\sum_{j=0}^{79} \frac{z^{2j+1}}{2j+1} \tag{19}$$

5.2 Trigonometry and complex values

Sine and cosine reduce the angle and keep 24 terms from the following series. Tangent divides sine by cosine. Arctangent uses a transformed argument u with small magnitude, then restores the angle offset and sign.

$$\sin x = \sum_{j=0}^{\infty} \frac{(-1)^j x^{2j+1}}{(2j+1)!}, \qquad \cos x = \sum_{j=0}^{\infty} \frac{(-1)^j x^{2j}}{(2j)!} \tag{20}$$

$$\arctan u \approx \sum_{j=0}^{63} \frac{(-1)^j u^{2j+1}}{2j+1} \tag{21}$$

For complex values, let z = a + ib. Its principal logarithm combines the logarithm of its magnitude with its angle. The complex power then follows from the exponential identity:

$$\log z = \ln |z| + i \arg z, \qquad |z| = \sqrt{a^2 + b^2}, \qquad z \neq 0 \tag{22}$$

$$e^{a+ib} = e^{a}(\cos\ b + i\ \sin\ b), \qquad z^{w} = \exp(w\ \log\ z) \tag{23}$$

6 Numerical methods and constants

A centred difference estimates a derivative. Euler's method updates the solution of a first-order differential equation one step at a time. Smaller steps can reduce truncation error, although floating-point effects still remain.

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \qquad y_{k+1} = y_k + hF(x_k, y_k) \tag{24}$$

Simpson integration combines alternating weights across an even number of intervals. Polynomial roots are estimated together using the Durand–Kerner update, where the leading coefficient is a_n. Repeated roots may not converge reliably.

$$h = \frac{b - a}{N}, \qquad x_j = a + jh, \qquad f_j = f(x_j) \tag{25}$$

$$I \approx \frac{h}{3}[f_0 + f_N + 4\sum_{j=1,3,\ldots}^{N-1} f_j + 2\sum_{j=2,4,\ldots}^{N-2} f_j] \tag{26}$$

$$z_i \leftarrow z_i - \frac{P(z_i)}{a_n\prod_{j\neq i}(z_i - z_j)} \tag{27}$$

6.1 Approximating pi

The pi routine repeatedly brings an arithmetic mean and a geometric mean together. Starting values are applied first, then the four stored values are updated together. The final ratio gives the approximation after the selected number of iterations.

$$a_0 = 1, \qquad b_0 = \frac{1}{\sqrt{2}}, \qquad t_0 = \frac{1}{4}, \qquad p_0 = 1 \tag{28}$$

$$a_{k+1} = \frac{a_k + b_k}{2}, \qquad b_{k+1} = \sqrt{a_k b_k} \tag{29}$$

$$t_{k+1} = t_k - p_k(a_k - a_{k+1})^2, \qquad p_{k+1} = 2\,p_k \tag{30}$$

$$\pi \approx \frac{(a_k + b_k)^2}{4\,t_k} \tag{31}$$

These routines extend the same method used for unsigned powers: store a small set of values, update them using a defined rule, and stop at a stated limit. An iterative approximation must still be checked for its domain, range and convergence.

7 Methodology and verification

The development method is to isolate one stage, check its expected behaviour, then connect it to the next stage. This keeps the calculation separate from the interface and makes errors easier to reproduce.

Start with the numeric types and unsigned checks. Add the exact and scientific power paths, then test their boundary cases. Next, connect the command reader to the expression parser, and the

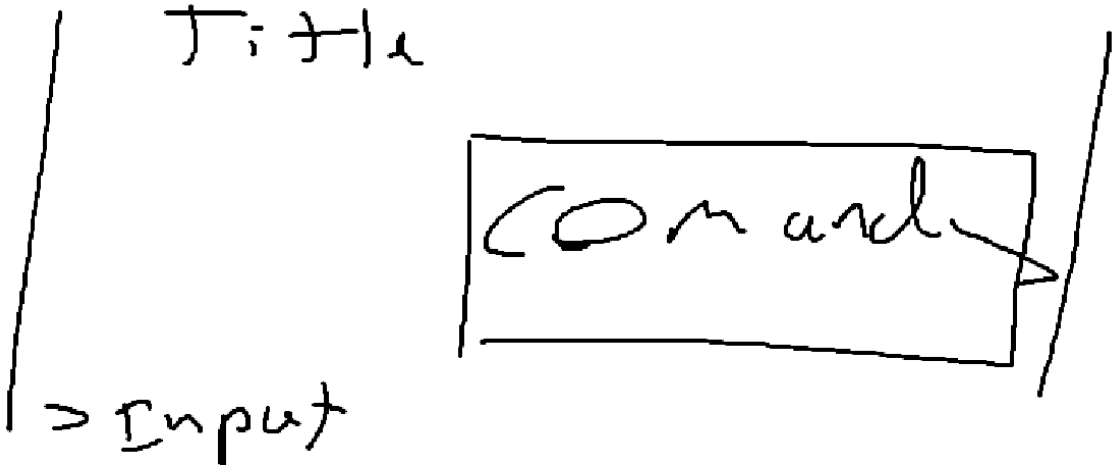
parser to the existing mathematics. Keep main.cpp responsible for startup, place implementation files in Source, and declarations in Header.

Open NoLibMath.sln in Visual Studio, select x64 / Release and run NoLibMath. Use NoLibMathLegacy for the original numbered menu. The original 94 source and header files remain unchanged; the new interface is added separately.

7.1 Check the behaviour

Check	Expected behaviour
Unsigned limit and one value above it	Accept the limit; reject the larger input.
A multiplication that exceeds storage	Stop the exact path before overflow.
The supplied cubic expression	Differentiate, then evaluate at a known value.
Missing bracket or incomplete power	Show a location-specific error and accept a new line.
Division by zero or an invalid logarithm	Return a domain error, not a numeric answer.

The delivered implementation passed 3,120 Cpp checks in both Debug and Release, plus 14 process and console checks in each configuration. These results confirm the tested behaviour; they do not mean every possible mathematical expression is supported.



7.2 Current limits and next step

Input is limited to 2,048 characters, 512 nodes and depth 48. Exponents must be constant integers from −1,024 to 1,024. Polynomial expansion supports degree 32. The workbench does not accept equations, extra variables or square-root syntax. Scientific notation increases the representable scale, not the number of exact digits.

To reproduce the project, begin with a small calculation that can be checked by hand. Confirm the input, the operation and the answer before increasing its complexity. This gives each new feature a clear reason for being added and a clear way to test whether it works.

> Implementation references: Source/NoLib/Core/CheckedInteger.cpp; Source/NoLib/Text/NumberParsing.cpp; Source/NoLib/Scalar; Source/Workbench; Header/NoLib/Symbolic and Numerical. Further numerical limitations are recorded in README.original.md.